

Site Guide

How to edit, extend and publish alecontri.github.io

September 2026

Contents

How the site is organized	1
Editing an existing page	1
Editing the homepage	2
Adding a brand-new simple page	2
Adding a new Rabbit Holes topic	2
Embedding images and video in the text	3
Math	3
Running the site locally	4
Publishing	4
Colors & fonts	4
A few loose ends worth knowing about	4

This is a short, practical reference for maintaining your own site – not a Jekyll manual. It covers exactly the patterns this site already uses. When in doubt, copy the closest existing file and adapt it; every page on the site was built that way.

How the site is organized

The site is a plain [Jekyll](#) project, hosted on GitHub Pages and built automatically by the GitHub Actions workflow in `.github/workflows/pages.yml` every time you push to `main`.

<code>_config.yml</code>	site-wide settings: title, tagline, nav order, social links
<code>index.markdown</code>	homepage (hero + "About" section)
<code>3_Research.markdown</code> , <code>5_RabbitHoles.markdown</code> , <code>6_Teaching.markdown</code> , <code>2_CV.markdown</code>	the four top-level pages
<code>rabbitholes_*.markdown</code>	the individual Rabbit Holes write-ups (linked from <code>5_RabbitHoles</code> 's cards)
<code>_posts/</code>	blog posts (not linked in the nav, still live at their URL + RSS)
<code>_layouts/</code>	the three page templates: default, home, page (+ post)
<code>_includes/</code>	reusable snippets: nav bar, footer, mesh graphics, <code><head></code>
<code>_sass/</code>	all styling, split by purpose (see "Colors & fonts" below)
<code>assets/</code>	images, the CV PDF, and per-topic media (<code>assets/numalg/</code>)

Every content page is a plain Markdown file with a **front matter** block at the top (between `---` lines) followed by the page's text. You can write normal Markdown, or drop in raw HTML – both work, and both are used throughout the site.

Editing an existing page

Open the relevant `.markdown` file and edit the text below the front matter. The front matter fields you'll see:

Field	Meaning
layout	Almost always <code>page</code> (or <code>home</code> for the homepage, <code>post</code> for blog posts). Controls which template wraps the content.
title	The big heading at the top of the page, and the browser-tab title.
eyebrow	The small label above the title (e.g. “Rabbit Holes”, “What I work on”). Optional.
permalink	The page’s URL, e.g. <code>/research/</code> . Keep these stable once published – changing one breaks any links or bookmarks pointing at the old URL.
back_url / back_label	Only used on the Rabbit Holes topic pages – adds a “back to Rabbit Holes” link at the top. Point <code>back_url</code> at the page you want to link back to.

Nothing else needs to change for a text edit – save the file, and (once you `git push`) the site rebuilds itself.

Editing the homepage

The homepage (`index.markdown`) has two parts:

1. **The hero** (name, role, tagline, portrait, the four buttons) – these come from variables at the top of `_config.yml`: `title`, `first_name`, `last_name`, `role`, `affiliation`, `tagline`. Edit them there, not in `index.markdown`.
2. **The “About” section** – this is simply the Markdown body of `index.markdown`. Edit that text directly; it’s wrapped in the “About” heading automatically by `_layouts/home.html`.

Adding a brand-new simple page

1. Copy an existing page close to what you want (`6_Teaching.markdown` is the simplest example) to a new file, e.g. `7_Publications.markdown`.
2. Update its front matter: `title`, `eyebrow`, and a unique `permalink` (e.g. `/publications/`).
3. Write the page content below the front matter.
4. **To add it to the top navigation**, add its filename to the `header_pages` list in `_config.yml`, in the position you want it to appear:

```
header_pages:
- 3_Research.markdown
- 6_Teaching.markdown
- 5_RabbitHoles.markdown
- 7_Publications.markdown  # <- new page
- 2_CV.markdown
```

If you *don’t* add it to `header_pages`, the page still exists and is reachable by its URL (useful for the Rabbit Holes topic pages, which are deliberately left out of the top nav and only reached via the Rabbit Holes cards).

Adding a new Rabbit Holes topic

This is the “card that opens into a full write-up” pattern used by the three current Rabbit Holes topics. It’s two pieces:

1. **The write-up page.** Copy one of the existing `rabbit_holes_*.markdown` files (`rabbit_holes_three_body.markdown` is a good template – it has math, a table, code, an image, and a video). Give it:

```

---
layout: page
title: Your topic title
eyebrow: Rabbit Holes
permalink: /rabbit-holes/your-topic-slug/
back_url: /rabbit-holes/
back_label: Rabbit Holes
---

```

Then write the content. Inline math uses $...$, display equations use
$$...$$
 (see “Math” below).

2. The card. Open `5_RabbitHoles.markdown` and copy one of the three `<a class="project-card">` blocks inside the `<div class="card-grid">`, then edit its href, image, title, description and tags to match your new page.

Embedding images and video in the text

All media lives under `assets/` – general site images directly in `assets/`, Rabbit Holes-specific media in `assets/numalg/`. Reference files with Jekyll’s `relative_url` filter so links keep working regardless of where the site is hosted.

A simple inline image (as used on the Rabbit Holes cards):

```



```

A captioned figure in article text (as used on the three Rabbit Holes write-ups) – wrap it in `<figure class="figure">` so it gets the site’s card-style border, shadow and caption styling:

```

<figure class="figure">
  
  <figcaption>A short caption explaining what's shown.</figcaption>
</figure>

```

An embedded video – same figure wrapper, with a `<video>` tag. `autoplay loop muted playsinline` makes it behave like a silent, self-looping clip rather than a full player with controls (recommended for short simulation clips); drop those four attributes if you want normal playback controls instead. A poster image is shown before the video loads:

```

<figure class="figure">
  <video src="{{ "/assets/numalg/your-clip.mp4" | relative_url }}"
    poster="{{ "/assets/numalg/your-clip-poster.jpg" | relative_url }}"
    autoplay loop muted playsinline></video>
  <figcaption>A short caption.</figcaption>
</figure>

```

Keep video files small (a few MB at most) – an easy way is `ffmpeg -i input.mov -vf scale=720:-1 -crf 26 -movflags +faststart output.mp4`. If you ever have a large GIF you want to embed as a simulation result, converting it to `.mp4/.webm` the same way will make the page load far faster than a raw GIF.

Math

Every page can use LaTeX math via MathJax: $inline$ and
$$...$$
. A handful of shorthand macros are predefined in `_layouts/default.html` – `\b{x}` for bold vectors, `\norm{x}`, `\inn{x}{y}` for inner products, `\RR` for $\mathbb{R}$, and a few more. Add new macros there if you find yourself repeating the same LaTeX often.

Running the site locally

```
bundle install
bundle exec jekyll serve
```

Then open `http://localhost:4000`. If `bundle install` complains about permissions, run `bundle config set --local path 'vendor/bundle'` first – this installs gems into a local, project-only folder instead of your system Ruby (already gitignored).

Publishing

Just commit and push to main:

```
git add -A
git commit -m "Update research page"
git push
```

The GitHub Actions workflow (`.github/workflows/pages.yml`) builds and deploys automatically – check the “Actions” tab on GitHub if a change doesn’t show up after a minute or two. This requires the repository’s **Settings > Pages > Build and deployment > Source** to be set to **GitHub Actions** (a one-time setup step, not something you need to touch again).

Colors & fonts

Everything visual is controlled from `_sass/_variables.scss` as CSS custom properties – change a value there and it updates everywhere, in both light and dark mode:

- `--accent` – the rust/orange accent color (buttons, links, highlights). There’s a separate, brighter value for dark mode right below it.
- `--bg`, `--card`, `--ink` – the paper background, card background, and text colors.
- `--font-display`, `--font-body`, `--font-mono` – the three fonts (Fraunces, Inter, IBM Plex Mono). Swap the Google Fonts `<link>` in `_includes/head.html` if you change these.

Dark mode follows the visitor’s system setting automatically, and the moon/sun toggle in the nav bar lets them override it (saved in their browser, per-visitor).

A few loose ends worth knowing about

- **Blog posts** (`_posts/`) still work exactly like normal Jekyll posts, and are still available via their URL and the RSS feed at `/feed.xml` – they’re just not linked from the nav or homepage anymore. Add a link to `/posts/...` manually from any page if you want to surface one again.
- **The CV page** embeds `assets/AlessandroContri_resume.pdf` directly – replace that file (keeping the same name) to update it, or change the filename in `2_CV.markdown` if you rename it.
- **404 page** lives at `404.html` in the repo root and follows the same mesh-themed styling as the rest of the site.